



UNIVERSITÀ
DEGLI STUDI
DI BERGAMO

Dipartimento
di Ingegneria Gestionale,
dell'Informazione e della Produzione

Problemi di soddisfacimento di vincoli

Ingegneria Informatica

Docente

Domenico Fabio Savo

Intelligenza Artificiale

Problema di soddisfacimento dei vincoli: introduzione

Fino ad ora abbiamo seguito l'idea che i problemi si possano risolvere eseguendo una ricerca nello **spazio degli stati**: un grafo in cui i nodi sono **stati** e gli archi che li collegano sono **azioni**.

Dal punto di vista degli algoritmi di ricerca ogni stato è **atomico**: una **scatola nera** di cui non consociamo la **struttura interna**.

Ora forniremo una **rappresentazione fattorizzata** per ogni stato, cioè un insieme di **variabili**, ognuna delle quali ha un **valore**.

- Un problema è risolto quando ogni variabile ha un valore che soddisfa **tutti i vincoli su di essa**.
- Un problema descritto in questo modo è detto **problema di soddisfacimento di vincoli**, o **CSP** (*constraint satisfaction problem*).



Algoritmi di ricerca CSP: introduzione

Gli algoritmi di ricerca CSP sfruttano la **struttura degli stati** (che ora è conosciuta) e utilizzano euristiche *di **uso generale*** (anziché *specifiche del dominio*) per individuare la soluzione di problemi complessi.

L'idea principale è quella di eliminare **ampie porzioni dello spazio di ricerca** tutte insieme, individuando combinazioni di variabili e valori che **violano** i vincoli.

I CSP hanno il vantaggio che le **azioni** e il **modello di transizione** possono essere dedotti direttamente dalla **descrizione del problema**.



Problemi CSP

Un problema **CSP** è costituito da tre componenti **X**, **D** e **C**, dove:

- **X** è un insieme di **variabili**, $\{X_1, \dots, X_n\}$.
- **D** è un insieme di **domini**, $\{D_1, \dots, D_n\}$, uno per ogni variabile.
- **C** è un insieme di **vincoli** che specificano combinazioni di valori ammesse.

Un dominio **D_i** è costituito da un insieme di valori ammessi, $\{v_1, \dots, v_k\}$, per la variabile **X_i**.

Esempio: una variabile **booleana** avrebbe il dominio $\{\text{vero}, \text{falso}\}$. Variabili diverse possono avere domini differenti e di dimensioni diverse.

Ogni vincolo **C_i** è costituito da una coppia $\langle \text{ambito}, \text{rel} \rangle$, dove **ambito** è una tupla di variabili che partecipano nel vincolo e **rel** è una relazione che definisce i valori che tali variabili possono assumere.

Una relazione può essere rappresentata come insieme esplicito di tutte le tuple di valori che soddisfano il vincolo, o come una funzione che descrive se una tupla è membro della relazione.

Esempio: se **X₁** e **X₂** hanno entrambe il dominio $\{1, 2, 3\}$, allora il vincolo che afferma che **X₁** deve essere maggiore di **X₂** si può scrivere come:

$\langle (X_1, X_2), \{(3, 1), (3, 2), (2, 1)\} \rangle$ oppure come $\langle (X_1, X_2), X_1 > X_2 \rangle$.



Soluzione di un problemi CSP

SOLUZIONE:

- Nei CSP si effettuano **assegnamenti di valori** a variabili in **X**.
Esempio: $\{X1 = v1, X2 = v2, \dots\}$
- Un assegnamento che **non viola alcun vincolo** è chiamato **consistente** o **legale**.
- Un assegnamento è **completo** se a tutte le variabili è assegnato un valore.
- Una **soluzione di un problema CSP** è un assegnamento **completo** e **consistente**.

SOLUZIONE PARZIALE:

- Un assegnamento è **parziale** se alcune delle variabili rimangono non assegnate.
- Una **soluzione parziale** è un assegnamento **parziale** e **consistente**.



Esempio di problema di soddisfacimento di vincoli: Map Coloring (1\3)

Supponiamo di dover colorare le regioni di una mappa (quella dell'Australia), caratterizzate da un nome, in modo tale che due regioni contigue siano colorate da colori diversi.

Supponiamo che i colori a disposizione siano rosso (**r**), verde (**v**) e blu (**b**).

- Definiamo una variabile per ogni regione:

$$X = \{WA, NT, Q, NSW, V, SA, T\}.$$

- Il dominio di ogni variabile è l'insieme $D_i = \{r, v, b\}$.
 - I vincoli richiedono che le regioni adiacenti abbiano colori distinti.
- Considerando la topografia mappa avremo:

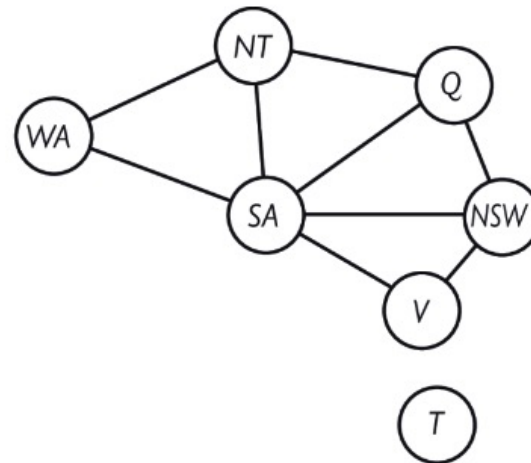
$$C = \{ \langle (SA, WA), SA \neq WA \rangle, \quad \langle (SA, NT), SA \neq NT \rangle, \quad \langle (SA, Q), SA \neq Q \rangle, \\ \langle (SA, NSW), SA \neq NSW \rangle, \quad \langle (SA, V), SA \neq V \rangle, \quad \langle (WA, NT), WA \neq NT \rangle, \\ \langle (NT, Q), NT \neq Q \rangle, \quad \langle (Q, NSW), Q \neq NSW \rangle, \quad \langle (NSW, V), NSW \neq V \rangle \}.$$



Esempio di problema di soddisfacimento di vincoli: Map Coloring (2\3)

Può essere utile visualizzare un CSP come un **grafo di vincoli** dove:

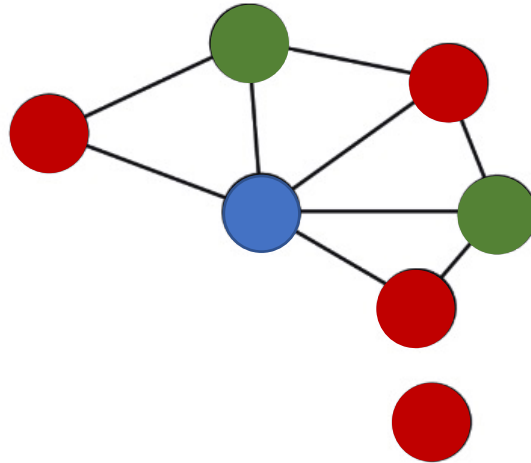
- i nodi del grafo corrispondono alle variabili del problema
- un arco connette ogni coppia di variabili **che partecipano a un vincolo**.



Esempio di problema di soddisfacimento di vincoli: Map Coloring (3\3)

Nel nostro esempio il problema ha diverse soluzioni. Una può essere la seguente:

$\{WA = \text{r}, NT = \text{v}, Q = \text{r}, NSW = \text{v}, V = \text{r}, SA = \text{b}, T = \text{r}\}$



Non è detto che per ogni mappa esista una soluzione al problema del Map Coloring associato

Esempio di problema di soddisfacimento di vincoli: Job-shop Scheduling (1\4)

Supponiamo di dover assemblare un'automobile.

Il lavoro è composto da vari compiti, ognuno dei quali è modellato da una variabile (il nostro insieme **X**)

Il valore che può essere assegnato ad una variabile (cioè il suo dominio) è il tempo al quale il compito inizia, espresso come numero intero di minuti.

I vincoli sono espressi per stabilire:

- che un compito deve essere svolto prima di un altro
- che solo un certo numero massimo di compiti possono essere svolti contemporaneamente
- che un compito richiede una certa quantità di tempo per essere portato a termine.



Esempio di problema di soddisfacimento di vincoli: Job-shop Scheduling (2\4)

Consideriamo una piccola parte dell'assemblaggio di un'auto, costituita da **14** compiti: installare gli assi (anteriore e posteriore), fissare tutte e quattro le ruote (destra e sinistra, anteriore e posteriore), stringere i dadi per ogni ruota e fissare i copriruota. Possiamo rappresentare questi compiti con 14 variabili:

$\mathbf{X} = \{ \text{AsseA, AsseP, RuotaDA, RuotaSA, RuotaDP, RuotaSP, DadiDA, DadiSA, DadiDP, DadiSP, CopriDA, CopriSA, CopriDP, CopriSP} \}.$

Ora rappresentiamo i **vincoli di precedenza** tra singoli compiti.

Ogni volta che un compito T_1 deve essere svolto prima del compito T_2 , e il compito T_1 richiede la durata d_1 per essere portato a termine, aggiungiamo un vincolo aritmetico della forma:

$$T_1 + d_1 \leq T_2 \quad (\text{abbreviazione per } < (T_1, T_2), T_1 + d_1 \leq T_2 >)$$



Esempio di problema di soddisfacimento di vincoli: Job-shop Scheduling (3\4)

Nel nostro esempio, gli assi devono essere già al loro posto prima che le ruote siano fissate, e l'installazione di un asse richiede **10** minuti, perciò **C** conterrà i seguenti vincoli:

$$\begin{aligned} \text{AsseA} + 10 &\leq \text{RuotaDA}, \text{AsseA} + 10 \leq \text{RuotaSA}, \\ \text{AsseP} + 10 &\leq \text{RuotaDP}, \text{AsseP} + 10 \leq \text{RuotaSP} \end{aligned}$$

Ora diciamo che, per ogni ruota, dobbiamo fissare la ruota stessa (compito che richiede 1 minuto), poi stringere i dadi (2 minuti) e infine installare il copriruota:

$$\begin{aligned} \text{RuotaDA} + 1 &\leq \text{DadiDA}; & \text{DadiDA} + 2 &\leq \text{CopriDA}; \\ \text{RuotaSA} + 1 &\leq \text{DadiSA}; & \text{DadiSA} + 2 &\leq \text{CopriSA}; \\ \text{RuotaDP} + 1 &\leq \text{DadiDP}; & \text{DadiDP} + 2 &\leq \text{CopriDP}; \\ \text{RuotaSP} + 1 &\leq \text{DadiSP}; & \text{DadiSP} + 2 &\leq \text{CopriSP}. \end{aligned}$$



Esempio di problema di soddisfacimento di vincoli: Job-shop Scheduling (4\4)

Supponiamo, inoltre, di avere quattro lavoratori per installare ruote, che però devono condividere un solo strumento per facilitare il posizionamento dell'asse. Ci serve un vincolo disgiuntivo che affermi che AsseA e AsseP non devono sovrapporsi nel tempo; uno dei due deve venire prima dell'altro:

$(\text{AsseA} + 10 \leq \text{AsseP})$ **or** $(\text{AsseP} + 10 \leq \text{AsseA})$.

Infine, supponiamo che vi sia un requisito per cui l'intero assemblaggio debba essere svolto in **30** minuti. Possiamo esprimere tale vincolo limitando il dominio di tutte le variabili:

$D_i = \{1, 2, 3, \dots, 30\}$



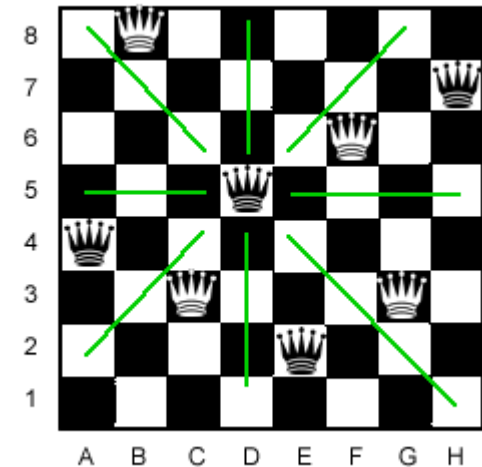
Esempio di problema di soddisfacimento di vincoli: Le 8 regine (1\3)

Problema: Inserire 8 regine in una scacchiera 8x8 in modo che non si attacchino. Una posizione è indicata con la coppia $[r, c]$ dove r è il numero di riga e c il numero di colonna.

Alla soluzione del problema si dedicò anche il noto matematico C. F. Gauss che trovò 72 diverse soluzioni. Le soluzioni possibili sono in realtà 92.

Per formalizzare il problema come CSP dobbiamo identificare:

- un insieme di variabili **X**
- un insieme di domini **D**
- un insieme di vincoli **C**



Esempio di problema di soddisfacimento di vincoli: Le 8 regine (2\3)

Dato che su ogni colonna potrà esserci al più una regina (altrimenti si attaccherebbero vicenda), possiamo formalizzare il problema con 8 variabili (una per ogni colonna)

$$X = \{ X_1, X_2, X_3, X_4, X_5, X_6, X_7, X_8 \}$$

Il pedice indica la colonna mentre l'assegnazione di un valore ad una variabile invece indica la riga in cui è posizionata una regina ($X_2 = 5$ indica che nella posizione $[5, 2]$ c'è una regina)

Quindi per ogni variabile il dominio è una delle 8 possibili righe:

$$D_1, = D_2, = D_3 \dots = D_8 = \{1,2,3,4,5,6,7,8\}$$



Esempio di problema di soddisfacimento di vincoli: Le 8 regine (3\3)

L'insieme dei vincoli **C** conterrà i seguenti vincoli per ogni coppia di colonne **(i,j)** con **i ≠ j**.

$$X_i \neq X_j$$

$$X_i \neq X_j + (j - i)$$

$$X_i \neq X_j - (j - i)$$

Il primo vincolo indica che due regine non possono trovarsi **sulla stessa riga**

I restanti vincoli impongono che il posizionamento di due regine sulla **stessa diagonale** sia scartato in quanto soluzione non ammissibile.

Nota: la definizione stessa del problema esclude il caso in cui due regine si trovino sulla stessa colonna



Tipi di vincolo

Il tipo di vincolo più semplice è quello **unario** che interessa il valore di una singola variabile. Ad esempio potremmo imporre che una regione nel problema del Map Coloring non può essere colorata di Verde:

Es: $SA \neq v$

I vincoli visti nei precedenti esempi sono invece **binari** poiché mettono in relazione due variabili.

Es: $X_i \neq X_j$

Un vincolo che interessa un numero arbitrario di variabili si dice **vincolo globale**. Il classico esempio di vincolo globale è il **TutteDiverse** che impone che tutte le variabili interessate dal vincolo debbano avere valori diversi.



Il Sudoku come CSP (esempio vincolo TutteDiverse)

Si parte da una griglia 9×9 in cui alcune caselle contengono dei valori (interi da 1 a 9). Il gioco consiste nel riempire tutte le caselle rimanenti con interi compresi tra 1 e 9 facendo in modo che nessuna cifra compaia due volte in una riga, colonna o riquadro 3×3 . Una riga, una colonna o un riquadro è chiamata **unità**.

	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

	1	2	3	4	5	6	7	8	9
A	4	8	3	9	2	1	6	5	7
B	9	6	7	3	4	5	8	2	1
C	2	5	1	8	7	6	4	9	3
D	5	4	8	1	3	2	9	7	6
E	7	2	9	5	6	4	1	3	8
F	1	3	6	7	9	8	2	4	5
G	3	7	2	6	8	9	5	1	4
H	8	1	4	2	5	3	7	6	9
I	6	9	5	4	1	7	3	8	2

Esempio di Sudoku con la sua soluzione



Il Sudoku come CSP (esempio vincolo TutteDiverse)

- Una istanza di Sudoku può essere considerata un CSP con 81 variabili, una per ogni casella.
- Utilizziamo i nomi di variabili da **A1** ad **A9** per la riga più in alto (da sinistra a destra), fino a **I1** – **I9** per la riga più in basso.
- Le caselle vuote hanno il dominio $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$ e quelle già riempite hanno un dominio costituito da un singolo valore.
- In più, vi sono **27** diversi vincoli **TutteDiverse**: uno per ciascuna riga, colonna e riquadro di 9 caselle.

Esempio:

TutteDiverse(A1, A2, A3, A4, A5, A6, A7, A8, A9)

...

TutteDiverse(A1, A2, A3, B1, B2, B3, C1, C2, C3)

	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

Propagazione dei vincoli

Un algoritmo CSP (cioè un algoritmo per la risoluzione di un CSP) può adottare principalmente due strategie:

- generare successori scegliendo un nuovo assegnamento di variabile,
- oppure può effettuare un tipo specifico di **inferenza** denominato **propagazione dei vincoli**: si usano i vincoli per ridurre il numero di valori legali per una variabile, il che a sua volta può ridurre i valori legali per un'altra variabile e così via.

Il concetto fondamentale alla base della propagazione dei vincoli è la **consistenza locale**. Se consideriamo ogni variabile come un nodo in un grafo e ogni vincolo binario come un arco, il processo di forzare la consistenza locale in ogni parte del grafo causa l'eliminazione dei valori inconsistenti in tutto il grafo.

Esistono diversi tipi di **consistenza locale**, che trattiamo nel seguito.



Consistenza di un Nodo

Una singola variabile (corrispondente a un nodo nel grafo CSP) è **nodo-consistente** se tutti i valori del suo dominio soddisfano i suoi **vincoli unari**.

Per esempio, se nella variante del problema di Map Coloring in cui la regione SA non può essere verde, se la variabile SA inizia con il dominio $\{\text{rosso}, \text{verde}, \text{blu}\}$, possiamo renderla **nodo-consistente** eliminando il *verde*, lasciandole quindi il dominio ridotto $\{\text{rosso}, \text{blu}\}$.

- Diciamo che un **grafo (o CSP) è nodo-consistente** se ogni sua **variabile è nodo-consistente**.

È facile eliminare tutti i vincoli unari in un CSP riducendo il dominio delle variabili con vincoli unari all'inizio del processo di risoluzione.



Consistenza d'Arco

Una **variabile** in un CSP si dice **arco-consistente** se ogni **valore del suo dominio** soddisfa i vincoli binari a cui partecipa.

- **Formalmente:** Una variabile X_i è **arco-consistente** rispetto a un'altra variabile X_j se per ogni valore nel dominio corrente D_i c'è almeno un valore nel dominio D_j che soddisfa il **vincolo binario** sull'arco (X_i, X_j) .
- Un **grafo è arco-consistente** se per ogni arco (X_i, X_j) , la variabile X_i è **arco-consistente** rispetto ad X_j e **viceversa**.

Esempio: consideriamo il vincolo $Y = X^2$ dove il dominio di X e di Y sono gli interi da 0 a 10. Possiamo scrivere esplicitamente tale vincolo come:

$\langle (X, Y), \{(0, 0), (1, 1), (2, 4), (3, 9)\} \rangle$

Per rendere X arco-consistente rispetto a Y , riduciamo il dominio di X a $\{0, 1, 2, 3\}$. Per rendere anche Y arco-consistente rispetto a X , riduciamo il suo dominio a $\{0, 1, 4, 9\}$. In questo caso l'intero CSP diventa arco-consistente.



Controllare la consistenza d'Arco: l'algoritmo AC-3

AC-3 è il più noto algoritmo per verificare la **consistenza d'arco** di CPS.

Per rendere ogni variabile **arco-consistente**, l'algoritmo mantiene un insieme **Arc** di archi (**orientati**) da considerare. Inizialmente **Arc** contiene tutti gli archi del CSP. AC-3 poi estrae e rimuove un arco orientato (X_i, X_j) da **Arc** (l'ordine di estrazione non è importante) e rende X_i **arco-consistente** rispetto a X_j . Se questo lascia invariato D_i , l'algoritmo passa all'arco successivo; se invece D_i cambia (riducendosi), aggiungiamo ad **Arc** tutti gli archi (X_k, X_i) .

Se D_i è ridotto all'insieme vuoto, sappiamo che l'intero problema CSP **non ha una soluzione consistente**. Altrimenti **continuiamo finché non vi sono più archi nella coda**.

AC-3 restituisce un CSP che è equivalente all'originale (con le stesse soluzioni), ma con domini di variabile più piccoli.

In alcuni casi, il problema si **risolve** (riducendo ogni dominio alla dimensione **1**) e in altri si dimostra che **non esiste soluzione consistente** (svuotando almeno un dominio).



L'algoritmo AC-3

```
function AC-3(csp) returns false se viene trovata una inconsistenza, o altrimenti true  
  Arc  $\leftarrow$  un insieme di archi orientati, inizialmente tutti quelli nel csp  
  while Arc non è vuota do  
     $(X_i, X_j) \leftarrow$  Estrai(Arc)  #Estrai e rimuovi un arco da Arc  
    if Rimuovi-Valori-Inconsistenti(csp,  $X_i, X_j$ ) then  
      if dimensione di  $D_i = 0$  then return false  
      for each  $X_k$  in  $X_i$ .Adiacenti do  
        aggiungi  $(X_k, X_i)$  ad Arc  
  
  return true
```

```
function Rimuovi-Valori-Inconsistenti(csp,  $X_i, X_j$ ) returns true se e solo se viene rimosso un valore  
  rimosso  $\leftarrow$  false  
  for each  $x$  in  $D_i$  do  
    if nessun valore  $y$  in  $D_j$  permette a  $(x, y)$  di soddisfare il vincolo tra  $X_i$  e  $X_j$  then  
      rimuovi  $x$  da  $D_i$   
      rimosso  $\leftarrow$  true  
  
  return rimosso
```



Algoritmo AC-3: complessità di calcolo

La complessità dell'algoritmo **AC-3** può essere analizzata come segue.

Supponiamo di avere un CSP con n variabili (nodi), ognuna con dimensione del dominio non superiore a d .

Supponiamo inoltre di avere c vincoli binari (archi).

Ogni arco (X_k, X_i) può essere inserito in **Arc** soltanto d volte perché X_i ha al più d valori che si possono eliminare.

Il controllo della consistenza di un arco può essere svolto in un tempo $O(d^2)$ dato che il peggiore dei casi prevede che si debbano verificare d valori in D_i con altrettanti d valori in D_j .

Otteniamo, quindi, un tempo totale del caso peggiore $O(cd^3)$, quindi **polinomiale** nella dimensione dell'input.



Algoritmo AC-3: Esempio (1/9)

Consideriamo il seguente CSP:

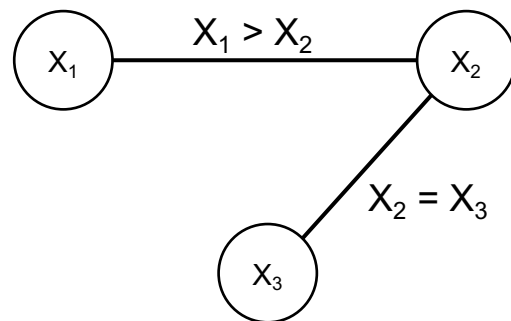
$$X = \{X_1, X_2, X_3\}$$

$$D = \{D_1 = \{1, 2, 3\}, D_2 = \{1, 2, 3\}, D_3 = \{1, 2, 3\}\}$$

$$C = \{X_1 > X_2,$$

$$X_2 = X_3 \}$$

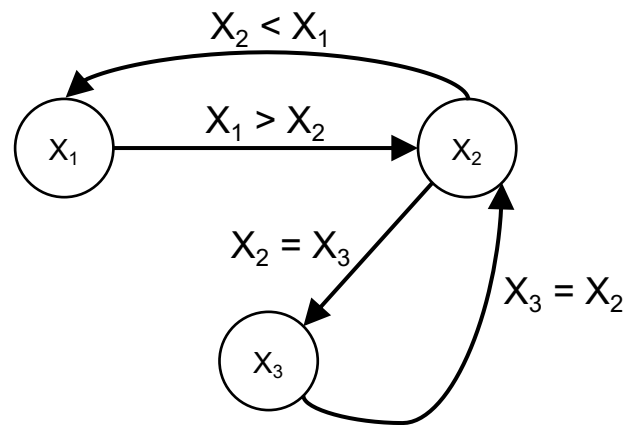
Il **grafo di vincoli** corrispondente sarà:



Archi
$X_1 > X_2$
$X_2 = X_3$

Algoritmo AC-3: Esempio (2/9)

Per applicare AC-3 dobbiamo considerare vincoli binari come archi *orientati*.
Otteniamo quindi:



Archi

$X_1 > X_2$

$X_2 < X_1$

$X_2 = X_3$

$X_3 = X_2$

La variabile di sinistra nel vincolo rappresenta la nostra X_i , mentre quella di destra la nostra X_j

Algoritmo AC-3: Esempio (3/9)

$$\begin{aligned} \mathbf{Arc} = \{ & X_1 > X_2, \\ & X_2 < X_1, \\ & X_2 = X_3, \\ & X_3 = X_2 \} \end{aligned} \quad D_1 = \{1, 2, 3\}, \quad D_2 = \{1, 2, 3\}, \quad D_3 = \{1, 2, 3\}$$

Estrai(**Arc**) $\rightarrow X_1 > X_2$

Rimuovi-Valori-Inconsistenti(csp, X_1, X_2) $\rightarrow D_1 = \{\textcolor{red}{4}, 2, 3\}, D_2 = \{1, 2, 3\}, D_3 = \{1, 2, 3\}$

Il dominio di X_1 è cambiato, l'unico arco entrante in X_1 è $X_2 < X_1$ che è già in **Arc**

$$\begin{aligned} \mathbf{Arc} = \{ & X_2 < X_1, \\ & X_2 = X_3, \\ & X_3 = X_2 \} \end{aligned}$$

Algoritmo AC-3: Esempio (4/9)

$$\begin{aligned} \mathbf{Arc} = \{ & X_2 < X_1, \\ & X_2 = X_3, \\ & X_3 = X_2 \} \end{aligned} \quad D_1 = \{2, 3\}, \quad D_2 = \{1, 2, 3\}, \quad D_3 = \{1, 2, 3\}$$

Estrai(**Arc**) $\rightarrow$ $X_2 < X_1$

Rimuovi-Valori-Inconsistenti(csp, X_2 , X_1) $\rightarrow$ $D_1 = \{2, 3\}$, $D_2 = \{1, 2, \textcolor{red}{3}\}$, $D_3 = \{1, 2, 3\}$

Il dominio di X_2 è cambiato: gli archi entranti in X_2 sono $X_3 = X_2$ (già presente) e $\textcolor{blue}{X_1} > \textcolor{blue}{X_2}$ (che aggiungo and **Arc**)

$$\begin{aligned} \mathbf{Arc} = \{ & X_2 = X_3, \\ & X_3 = X_2, \\ & \textcolor{blue}{X_1} > \textcolor{blue}{X_2} \} \end{aligned}$$



Algoritmo AC-3: Esempio (5/9)

$$\mathbf{Arc} = \{ \begin{array}{l} X_2 = X_3, \\ X_3 = X_2, \\ X_1 > X_2 \end{array} \} \quad D_1 = \{2, 3\}, \quad D_2 = \{1, 2\}, \quad D_3 = \{1, 2, 3\}$$

Estrai(**Arc**) $\rightarrow X_2 = X_3$

Rimuovi-Valori-Inconsistenti(csp, X_2, X_3) $\rightarrow D_1 = \{2, 3\}, D_2 = \{1, 2\}, D_3 = \{1, 2, 3\}$

Il dominio di X_2 non è cambiato quindi non aggiungo archi ad **Arc**

$$\mathbf{Arc} = \{ \begin{array}{l} X_3 = X_2, \\ X_1 > X_2 \end{array} \}$$



Algoritmo AC-3: Esempio (6/9)

$$\mathbf{Arc} = \{ X_3 = X_2, \quad D_1 = \{2, 3\}, \quad D_2 = \{1, 2\}, \quad D_3 = \{1, 2, 3\} \\ X_1 > X_2 \}$$

Estrai(**Arc**) $\rightarrow X_3 = X_2$

Rimuovi-Valori-Inconsistenti(csp, X_3, X_2) $\rightarrow D_1 = \{2, 3\}, D_2 = \{1, 2\}, D_3 = \{1, 2, \textcolor{red}{3}\}$

Il dominio di X_3 è cambiato: l'unico arco entrante in X_3 è $\textcolor{blue}{X_2} = \textcolor{blue}{X_3}$ che viene aggiunto ad **Arc**

$$\mathbf{Arc} = \{ X_1 > X_2, \\ \textcolor{blue}{X_2} = \textcolor{blue}{X_3} \}$$

Algoritmo AC-3: Esempio (7/9)

$$\mathbf{Arc} = \{ X_1 > X_2, \quad X_2 = X_3 \} \quad D_1 = \{2, 3\}, \quad D_2 = \{1, 2\}, \quad D_3 = \{1, 2\}$$

Estrai(**Arc**) $\rightarrow X_1 > X_2$

Rimuovi-Valori-Inconsistenti(csp, X_1, X_2) $\rightarrow D_1 = \{2, 3\}, D_2 = \{1, 2\}, D_3 = \{1, 2\}$

Il dominio di X_1 non è cambiato: non aggiunto archi ad **Arc**

$$\mathbf{Arc} = \{ X_2 = X_3 \}$$

Algoritmo AC-3: Esempio (8/9)

$$\mathbf{Arc} = \{X_2 = X_3\} \quad D_1 = \{2, 3\}, \quad D_2 = \{1, 2\}, \quad D_3 = \{1, 2\}$$

Estrai(**Arc**) $\rightarrow X_2 = X_3$

Rimuovi-Valori-Inconsistenti(csp, X_2, X_3) $\rightarrow D_1 = \{2, 3\}, D_2 = \{1, 2\}, D_3 = \{1, 2\}$

Il dominio di X_2 non è cambiato: non aggiunto archi ad **Arc**

$$\mathbf{Arc} = \{ \}$$



Algoritmo AC-3: Esempio (9/9)

$Arc = \{\}$ $D_1 = \{2, 3\}, D_2 = \{1, 2\}, D_3 = \{1, 2\}$

L'insieme **Arc** non contiene più archi e nessuno dei domini è vuoto, l'algoritmo termina restituendo *true*



Consistenza di cammino

Consideriamo il problema del Map Coloring.

In questo caso la consistenza d'arco non è molto utile, perché ogni variabile è già arco-consistente: per ogni nodo esisterà sempre un colore diverso per il nodo dall'altra parte dell'arco.

La consistenza d'arco restringe i domini (vincoli unari) utilizzando gli archi (vincoli binari). Per fare progressi in problemi quali il Map Coloring serve una nozione di consistenza più forte.

La **consistenza di cammino** restringe i vincoli binari utilizzando vincoli impliciti che sono inferiti considerando triplette di variabili.



Consistenza di cammino

Un insieme di due variabili $\{X_i, X_j\}$ è **cammino-consistente rispetto a una terza variabile X_m** se, per ogni assegnamento $\{X_i = a, X_j = b\}$ consistente con i vincoli (se esistono) su $\{X_i, X_j\}$, esiste un assegnamento di X_m che soddisfa i vincoli su $\{X_i, X_m\}$ e $\{X_m, X_j\}$.

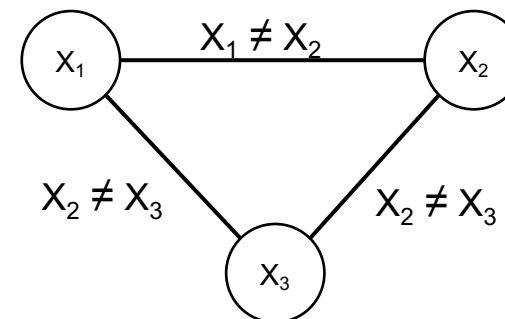
Il termine **consistenza di cammino** si riferisce alla consistenza complessiva del cammino da X_i a X_j con X_m nel mezzo.

Esempio:

$$\mathbf{X} = \{X_1, X_2, X_3\}$$

$$\mathbf{D} = \{D_1 = D_2 = D_3 = \{A, B\}\}$$

$$\mathbf{C} = \{X_1 \neq X_2, X_1 \neq X_3, X_2 \neq X_3\}$$



E' facile verificare che ogni nodo è arco consistente mentre qualunque coppia di nodi non può essere cammino-consistente rispetto al terzo nodo.

K-Consistenza

Forme più potenti di propagazione si possono definire usando il concetto di **k-consistenza**.

Un CSP è **k-consistente** se, per ogni insieme di **k-1** variabili e ogni loro assegnamento consistente, è sempre possibile assegnare un valore consistente a ogni **k-esima** variabile.

- La **1-consistenza** significa che, dato l'insieme vuoto, possiamo rendere consistente ogni insieme di una sola variabile: è il caso della **consistenza di nodo**.
- La **2-consistenza** corrisponde alla **consistenza d'arco**.
- La **3-consistenza** corrisponde alla **consistenza di cammino**.

Un CSP è detto **fortemente k-consistente** se è k -consistente e anche $(k-1)$ -consistente, $(k-2)$ -consistente, . . . fino a 1-consistente.



Ricerca della soluzione nei CSP

Spesso, dopo la conclusione del processo di propagazione dei vincoli (ad esempio dopo l'applicazione dell'algoritmo AC-3) rimangono ancora variabili con più valori possibili. Non abbiamo quindi ancora trovato una soluzione.

È quindi necessario effettuare una **ricerca della soluzione**.

Vedremo di seguito gli algoritmi di ricerca con **backtracking**



Ricerca con backtracking per CSP

L'idea alla base della ricerca con backtracking è quella di iniziare da un CSP senza assegnamenti e poi effettuare degli assegnamenti in modo ricorsivo fino ad individuare una soluzione o il fallimento.

I passi principali dell'algoritmo si possono riassumere come segue:

- Si sceglie ripetutamente una variabile a cui non è ancora stato assegnato un valore e si prova ad assegnare uno a uno tutti i valori del suo dominio, cercando di estendere ogni assegnamento a una soluzione attraverso una chiamata ricorsiva.
- Se la chiamata ha successo, viene restituita la soluzione, mentre se fallisce, l'assegnamento è riportato allo stato precedente e si prova il valore successivo.
- Se non si trova alcun valore che funziona, viene restituito fallimento.



Algoritmo di ricerca con backtracking per CSP

```
function Backtracking(csp, assegnamento) returns una soluzione, o fallimento
  if assegnamento è completo then return assegnamento
  var ← Scegli-Variabile-non-Assegnata(csp, assegnamento)
  for each valore in Ordina-Valori-Dominio(csp, var, assegnamento) do
    if valore è consistente con assegnamento then
      aggiungi {var = valore} ad assegnamento
      inferenze ← Inferenza(csp, var, assegnamento)
      if inferenze ≠ fallimento then
        aggiungi inferenze a csp
        risultato ← Backtracking(csp, assegnamento)
        if risultato ≠ fallimento then return risultato
      rimuovi inferenze da csp
    rimuovi {var = valore} da assegnamento
  return fallimento
```



Algoritmo di ricerca con backtracking per CSP

L'algoritmo ricorsivo **Backtracking** viene invocato dall'algoritmo **Ricerca-Backtracking** partendo da un assegnamento vuoto.

```
function Ricerca-Backtracking(csp) returns una soluzione, o fallimento  
    return Backtracking(csp, { })
```

- L'algoritmo **Backtracking** è modellato sulla **ricerca in profondità ricorsiva**.
- Le funzioni **Scegli-Variabile-non-Assegnata** e **Ordina-Valori-Dominio** sono usate per implementare delle **euristiche** di uso generale.
- La funzione **Inferenza** può essere usata, facoltativamente, per imporre **consistenza d'arco, di cammino o k -consistenza**.
- Se la scelta di un valore porta al **fallimento** (notato da Inferenza o Backtracking), allora gli assegnamenti di valori (inclusi quelli effettuati da Inferenza) sono annullati e si prova con un nuovo valore.



Uso di euristiche nell'algoritmo di backtracking

L'algoritmo **Backtracking** deve fare delle scelte che possono influenzare il numero azioni (assegnamenti) necessari per individuare la soluzione.

Tale scelte sono:

- **Quale variabile assegnare nel passo successivo?**: tale scelta è effettuata dalla funzione **Scegli-Variabile-Non-Assegnata**
- **In quale ordine provare i possibili valori da assegnare alle variabili?**: tale scelta è effettuata dalla funzione **Ordina-Valori-Dominio**.



Scelta delle variabili (Scegli-Variabile-Non-Assegnata)

L'algoritmo di ricerca con backtracking contiene l'istruzione:

$var \leftarrow \text{Scegli-Variabile-non-Assegnata}(csp, \text{assegnamento})$.

La strategia più semplice per **Scegli-Variabile-non-Assegnata** è scegliere le variabili secondo un ordine prestabilito. La seconda strategia più semplice è quella di scegliere le variabili a caso. → **Nessuna delle due strategie è ottima.**

Possiamo pensare a delle strategie più efficaci.



Scelta delle variabili: Minimum Remaining Values

L'idea alla base della euristica **MRV** (Minimum Remaining Values) è quella di scegliere la variabile con il **minor numero di valori legali** (che non violano nessun vincolo) rimanenti.

MRV è stata chiamata anche euristica “della variabile più vincolata” o *fail-first*, perché sceglie la variabile per cui è maggiore la probabilità di arrivare presto a un fallimento, potando così l'albero di ricerca.

- Se una variabile X_i non ha più alcun possibile valore, l'euristica MRV sceglierà X_i e il fallimento sarà rilevato immediatamente, evitando così ricerche inutili su altre variabili.

Sperimentalmente si è verificato che in media **MRV** è più efficiente di una scelta a caso delle variabili



Minimum Remaining Values: esempio

Supponiamo che l'algoritmo abbia processato i nodi WA, NT e Q (assegnando rispettivamente i valori red, green, red) e che ora debba scegliere la prossima variabile da processare:

Le variabili legali per ogni nodo non ancora assegnato sono:

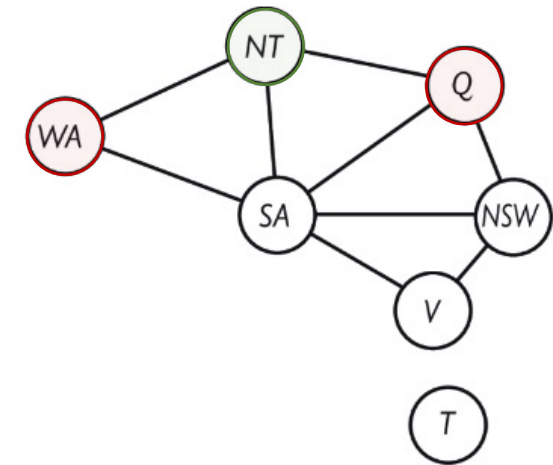
$$D_{SA} = \{\text{blu}\}$$

$$D_{NSW} = \{\text{green}, \text{blu}\}$$

$$D_V = \{\text{red}, \text{green}, \text{blu}\}$$

$$D_T = \{\text{red}, \text{green}, \text{blu}\}$$

Scegli-Variabile-non-Assegnata basata su **MRV** sceglierà SA come prossima variabile da processare



Scelta delle variabili: problema in MRV

La **MRV** non è di molto aiuto quando le variabili hanno ancora tutte, o quasi, valori legali nel loro dominio.

$$D_{WA} = \{\text{red}, \text{green}, \text{blue}\}$$

$$D_{NT} = \{\text{red}, \text{green}, \text{blue}\}$$

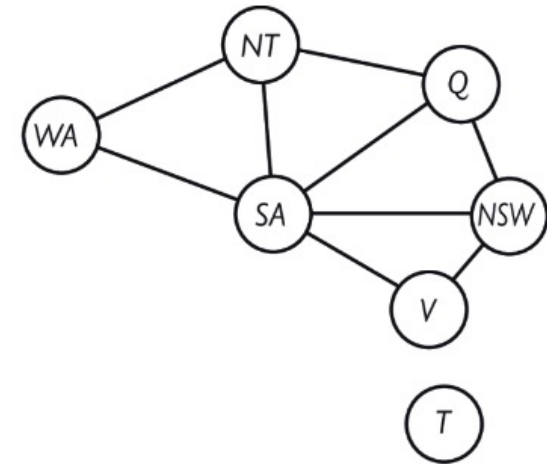
$$D_Q = \{\text{red}, \text{green}, \text{blue}\}$$

$$D_{SA} = \{\text{red}, \text{green}, \text{blue}\}$$

$$D_{NSW} = \{\text{red}, \text{green}, \text{blue}\}$$

$$D_V = \{\text{red}, \text{green}, \text{blue}\}$$

$$D_T = \{\text{red}, \text{green}, \text{blue}\}$$



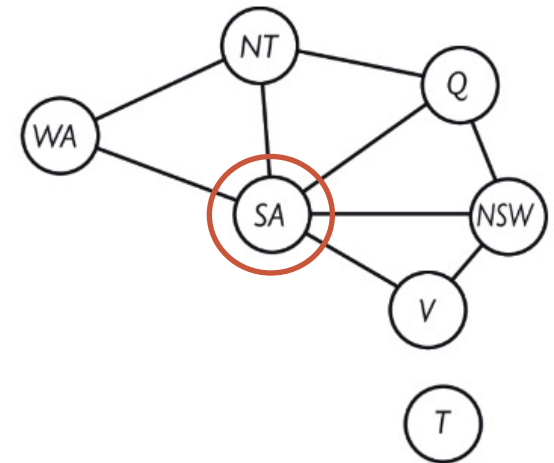
Non c'è una variabile con un numero minimo di valori residui! Come mi comporto?

Scelta delle variabili: euristica di grado

Quando **MRV** non è di aiuto (cioè quando non posso scegliere rispetto al numero minimo di valori residui) è d'aiuto l'**euristica di grado** (*degree heuristic*): questa euristica cerca di ridurre il fattore di ramificazione delle scelte future scegliendo la variabile **coinvolta nel maggior numero di vincoli**.

L'euristica **MRV** solitamente è più potente della **euristica di grado**, ma quest'ultima può tornare utile nel caso si presentino dei “pareggi” nell'ordinamento.

Nell'esempio, **SA** partecipa a **5** vincoli, **WA** e **V** a **2**, **NT**, **Q** e **NSW** a **3**, e **T** a **0**. L'**euristica di grado** sceglierà quindi **SA**.



Scelta dei valori (Scegli-Variabile-Non-Assegnata)

L'algoritmo di ricerca con backtracking contiene una chiamata alla funzione

Ordina-Valori-Dominio(*csp*, *var*, *assegnamento*) per scegliere i valori del dominio da assegnare ad una variabile.

Una strategia efficace è quella fornita dall'euristica del **valore meno vincolante** (*Least Constraining Value* (LCV)), che preferisce i valori che lasciano il più grande sottoinsieme di valori legali per le variabili non assegnate (numero di scelte possibili rispetto alle variabili adiacenti).

L'euristica LCV può permettere di arrivare più in fretta ad una soluzione: l'idea è che si danno maggiori possibilità alla ricerca di trovare futuri assegnamenti legali.

NOTA: Se una soluzione non c'è, o se vogliamo trovare tutte le soluzioni, l'euristica LCV non ha alcuna utilità.



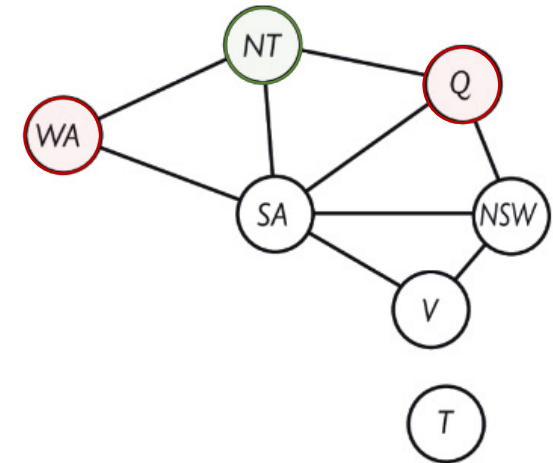
Least Constraining Value (LCV): esempio

Supponiamo che l'algoritmo abbia processato i nodi WA, NT assegnando rispettivamente i valori **red** e **green** e che la prossima variabile da processare sia Q.

Le variabili legali per Q sono:

$$D_Q = \{\text{red}, \text{blu}\}$$

Considerando i vincoli di SA e l'attuale assegnazione, se assegnassimo a Q il **blu** non riusciremmo ad individuare alcun valore legale per SA. Secondo l'euristica LCV sceglieremo quindi di assegnare a Q il valore **red**.



Verifica in avanti (Forward Checking)

AC-3 e altri algoritmi applicano tecniche di inferenza per ridurre i domini di variabili **prima** di iniziare la ricerca.

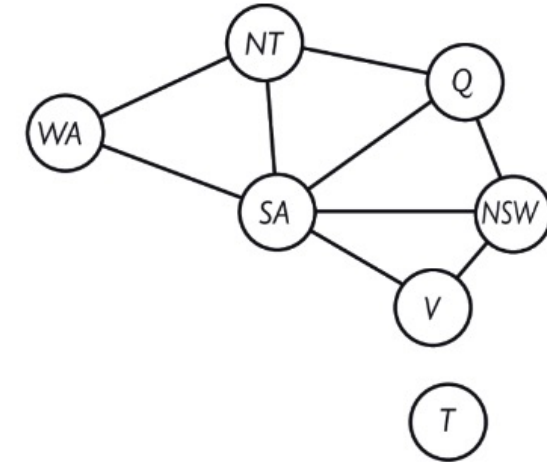
L'inferenza può risultare ancora più potente *nel corso* di una ricerca dato che ogni volta che scegliamo un valore per una variabile, abbiamo un'opportunità del tutto nuova di **inferire** nuove riduzioni dei domini delle variabili adiacenti.

Una delle più semplici forme di inferenza è la cosiddetta **verifica in avanti** o **forward checking (FC)**: Ogni volta che una variabile X è assegnata, il processo di verifica in avanti stabilisce la **consistenza d'arco per essa**: per ogni variabile non assegnata Y collegata a X da un vincolo, cancella dal dominio di Y ogni valore non consistente con quello scelto per X .



Forward Checking (FC): esempio

	WA	NT	Q	SA	NSW	V	T
Domini iniziali	{r,g,b}	{r,g,b}	{r,g,b}	{r,g,b}	{r,g,b}	{r,g,b}	{r,g,b}
dopo WA = r	→ r	{g,b}	{r,g,b}	{g,b}	{r,g,b}	{r,g,b}	{r,g,b}
dopo Q = g	→ r	{b}	→ g	{b}	{r,b}	{r,g,b}	{r,g,b}
dopo V = b	→ r	{b}	→ g	∅	{r}	→ b	{r,g,b}



Esempio: effettuiamo una ricerca con backtracking sul CSP dell'Australia con verifica in avanti. Senza applicare alcuna euristica di scelta delle variabili, effettuiamo le assegnazioni di WA, Q e V, in quest'ordine. La verifica in avanti ha rilevato che l'assegnamento parziale {WA = r, Q = v, V = b} è **inconsistente** con i vincoli del problema, e l'algoritmo potrà quindi tornare immediatamente indietro con il backtracking.

Alternanza di ricerca e inferenza

Per molti problemi la ricerca risulterà più efficace se combiniamo l'euristica **MRV** (Minimum Remaining Values), per la scelta delle variabili, con la **verifica in avanti**.

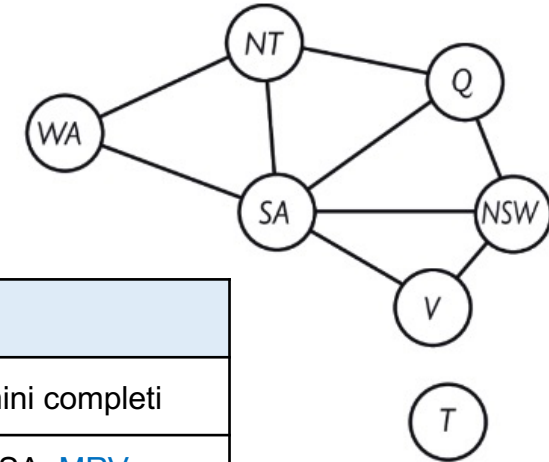
Proviamo a gestire la scelta delle variabili dell'esempio precedente in maniera più strategica:

1. Dopo il primo assegnamento $WA = r$, cerchiamo la prossima variabile con **MRV** (Minimum Remaining Values)
2. Immediatamente dopo ogni assegnamento, aggiorniamo i domini delle variabili con la **verifica in avanti**. L'idea è usare la **verifica in avanti** per calcolare in modo incrementale l'informazione di cui **MRV** necessita per fare il suo lavoro.



Alternanza di ricerca e inferenza: esempio

NOTA: Per questo esempio, effettuiamo la scelta del **valore** all'interno del dominio di una variabile seguendo l'ordinamento (r,g,b)



	WA	NT	Q	SA	NSW	V	T	Commento
domini iniziali	{r,g,b}	{r,g,b}	{r,g,b}	{r,g,b}	{r,g,b}	{r,g,b}	{r,g,b}	Tutte le variabili hanno domini completi
dopo WA = r (prima scelta fatta in modo arbitrario)	→ r	{g,b}	{r,g,b}	{g,b}	{r,g,b}	{r,g,b}	{r,g,b}	FC restringe i domini di NT e SA. MRV non sa scegliere tra i due, ma degree heuristic indica SA come prossima scelta
dopo SA = g	→ r	{b}	{r,b}	→ g	{r,b}	{r,b}	{r,g,b}	FC restringe i domini di NT, Q, NSW e V. MRV sceglie NT
dopo NT = b	→ r	→ b	{r}	→ g	{r,b}	{r,b}	{r,g,b}	FC restringe il dominio di Q e MRV lo sceglie
dopo Q = r	→ r	→ b	→ r	→ g	{b}	{r,b}	{r,g,b}	FC restringe il dominio di NSW e MRV lo sceglie
dopo NSW = b	→ r	→ b	→ r	→ g	→ b	{r}	{r,g,b}	FC restringe il dominio di V e MRV lo sceglie
dopo V = r	→ r	→ b	→ r	→ g	→ b	→ r	{r,g,b}	Il dominio di T è rimasto invariato. Assegniamo T = r seguendo l'ordinamento dei valori
dopo T = r	→ r	→ b	→ r	→ g	→ b	→ r	→ r	Soluzione!

Strategie di Backtracking

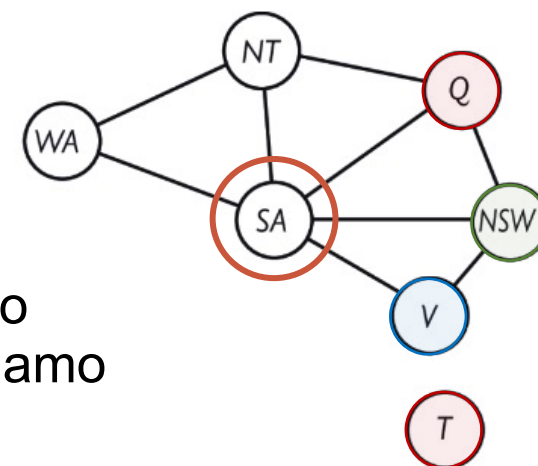
L'algoritmo **Backtracking** presentato, ha una politica molto semplice riguardo a cosa fare quando un ramo della ricerca fallisce:

- *tornare indietro alla variabile precedente e provare a usare un valore diverso. Questo si chiama **backtracking cronologico**, perché viene rivisitato il punto decisionale più recente.*

Questa strategia purtroppo non è sempre efficace

Esempio:

- Supponiamo che siano stati effettuati i seguenti assegnamenti in quest'ordine: $Q = r$, $NSW = g$, $V = b$, $T = r$.
- Supponiamo ora che sia il turno di SA. Come si può vedere il suo dominio non contiene variabili legali. Facendo backtracking torniamo alla mossa precedente e assegniamo un nuovo valore a T.
- Mossa inutile, il dominio di SA ancora non contiene valori legali!



Strategie di Backtracking: Conflict set

Un approccio più intelligente è quello di effettuare il backtracking **risalendo fino a una delle variabili che ha causato il fallimento**.

Per fare ciò usiamo l'**insieme dei conflitti (conflict set)** di variabile che contiene gli assegnamenti che sono in conflitto con qualche valore nel suo dominio.

Il **conflict set** di una variabile Y può essere costruito con il supporto di **Forward Chaining**:

- Ogni volta che **FC** causata da un assegnamento $X = x$ fa sì che venga cancellato un valore dal dominio di Y , andiamo ad aggiungere $X = x$ al **conflict set** di Y .
- Se il dominio di Y viene svuotato (i.e., abbiamo raggiunto una **contraddizione**), tutti gli assegnamenti del **conflict set** di Y devo essere aggiunti all'insieme dei conflitti di X .
- In questo modo, ora sappiamo che $X = x$ porta a una **contraddizione** (in Y) e quindi che occorre provare un diverso assegnamento per X .

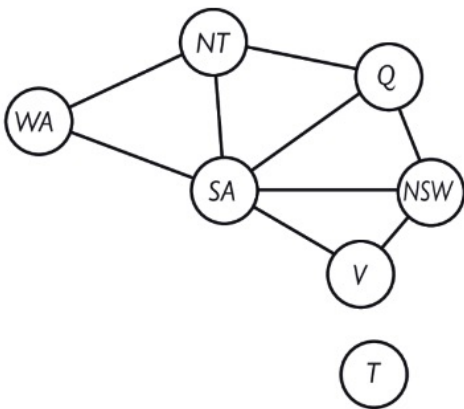


Strategie di Backtracking: Backjumping

Un volta che abbiamo il **conflict set** (CS) di una variabile possiamo, in caso di inconsistenza, tornare indietro nelle assegnazioni fino all'assegnazione che ha causato l'inconsistenza.

Tale operazione è detta **Backjumping** (salto all'indietro) e si distingue dal **backtracking cronologico** perché effettua dei salti indietro nell'albero decisionale anche di più passi.





Backjumping e conflict set: esempio

Ordine in cui le variabili vengono scelte per l'assegnamento:
(1) Q, (2) NSW, (3) V, (4) T, (5) SA

	WA	NT	Q	SA	NSW	V	T
domini iniziali	{r,g,b}	{r,g,b}	{r,g,b}	{r,g,b}	{r,g,b}	{r,g,b}	{r,g,b}
dopo Q = r	{r,g,b}	$\{g,b\}$ $cs = \{Q = r\}$	$\rightarrow r$	$\{g,b\}$ $cs = \{Q = r\}$	$\{g,b\}$ $cs = \{Q = r\}$	{r,g,b}	{r,g,b}
dopo NSW = g	{r,g,b}	$\{g,b\}$ $cs = \{Q = r\}$	$\rightarrow r$ $cs = \{NSW = g\}$	$\{b\}$ $cs = \{Q = r, NSW = g\}$	$\rightarrow g$ $Cs = \{Q = r\}$	$\{r,b\}$ $Cs = \{NSW = g\}$	{r,g,b}
dopo V = b	{r,g,b}	$\{g,b\}$ $cs = \{Q = r\}$	$\rightarrow r$ $cs = \{NSW = g\}$	$\emptyset$ $cs = \{Q = r, NSW = g, V = b\}$	$\rightarrow g$ $Cs = \{Q = r, V = b\}$	$\rightarrow b$ <u>$Cs = \{NSW = g, Q = r\}$</u>	{r,g,b}
dopo T = r	{r,g,b}	$\{g,b\}$ $cs = \{Q = r\}$	$\rightarrow r$ $cs = \{NSW = g\}$	$\emptyset$ $cs = \{Q = r, NSW = g, V = b\}$	$\rightarrow g$ $Cs = \{Q = r, V = b\}$	$\rightarrow b$ $Cs = \{NSW = g, Q = r\}$	$\rightarrow r$
SA = ??	Siamo bloccati perché il dominio di SA è vuoto. Si esegue il backjump per tornare indietro al più recente assegnamento del conflict set di SA, i.e., V = b						

